个税预测
技术文档

架构设计 • 算法实现 • 安全设计

项目名称	个税预测（Tax Forecast）v1.0.0
系统形态	纯前端单页应用，零后端 / 零构建 / 零依赖
代码规模	3,456 行（HTML 495 / CSS 1,174 / JS 1,787）
技术栈	原生 HTML + CSS + ES5 JavaScript
文档版本	v1.0	2026-10-27
适用读者	开发、测试、架构评审、答辩评审

本文档描述系统内部设计与实现细节，需求定义见《需求文档》，安全评估结论见《安全自评》。

目录

01 系统概述 定位、技术选型与关键取舍

02 总体架构 分层结构、模块职责、数据流

03 模块设计 四大模块的接口与实现要点

04 算法说明 计税、择优、盲区求解、预扣模拟、精确期权衡

05 数据模型 输入参数、计算结果、状态管理

06 安全设计 数据边界、凭据管理、内容安全、输入健壮性

07 性能与兼容性 渲染性能、响应式、浏览器矩阵

08 测试与验证 三层测试策略与数值断言

09 部署架构 静态托管与更新链路

10 演进规划 技术债与版本路线

01 系统概述

一句话定义：把中国个人所得税的「累计预扣 + 汇算清缴」两套口径，压缩成一个可以在 1 秒内跑完、且每个数字都能被逐项验证的浏览器端计算器。

1.1 技术选型的三条硬约束

这个产品的用户是普通纳税人，使用场景高度随机——可能在地铁上打开一个微信链接。因此技术选型不是「能用什么」，而是「不能依赖什么」。

约束	排除的方案	根本原因
零后端	Node/Java 服务端计算	税务数据属于个人敏感信息。一旦上服务器，就要面对存储加密、访问审计、跨境传输等一整套合规成本；而不上传，这些风险直接归零。
零构建	Webpack / Vite / TypeScript 编译	交付物是「双击 index.html 就能用」的目录。没有构建步骤，就没有「构建产物与源码不一致」的审计盲区，也便于评审方直接读源码。
零依赖	React、Chart.js、dayjs、任何 CDN	三个理由：断网可用、供应链攻击面为零、十年后仍能运行。图表因此用原生 DOM + CSS 手写，而非引入图表库。

1.2 关键取舍

取舍 1

ES5 而非现代语法

放弃箭头函数、可选链、模块化带来的开发效率，换取在旧版企业浏览器、微信内置 WebView 中的直接可用性，且无需 Babel。代价是代码稍长（1,787 行 JS）。

取舍 2

手写图表而非图表库

月度现金流只需 12 根堆叠柱，用 flex + 百分比高度 30 行代码即可实现，且天然支持打印与无 JS 环境降级。引入 Chart.js（约 200KB）在收益上不成立。

取舍 3

AI 能力可选而非内置

不内置任何厂商 Key。用户自带 OpenAI 兼容接口，未配置时降级为内置规则引擎。避免了密钥托管这一最高危风险，也避免了模型费用转嫁。

取舍 4

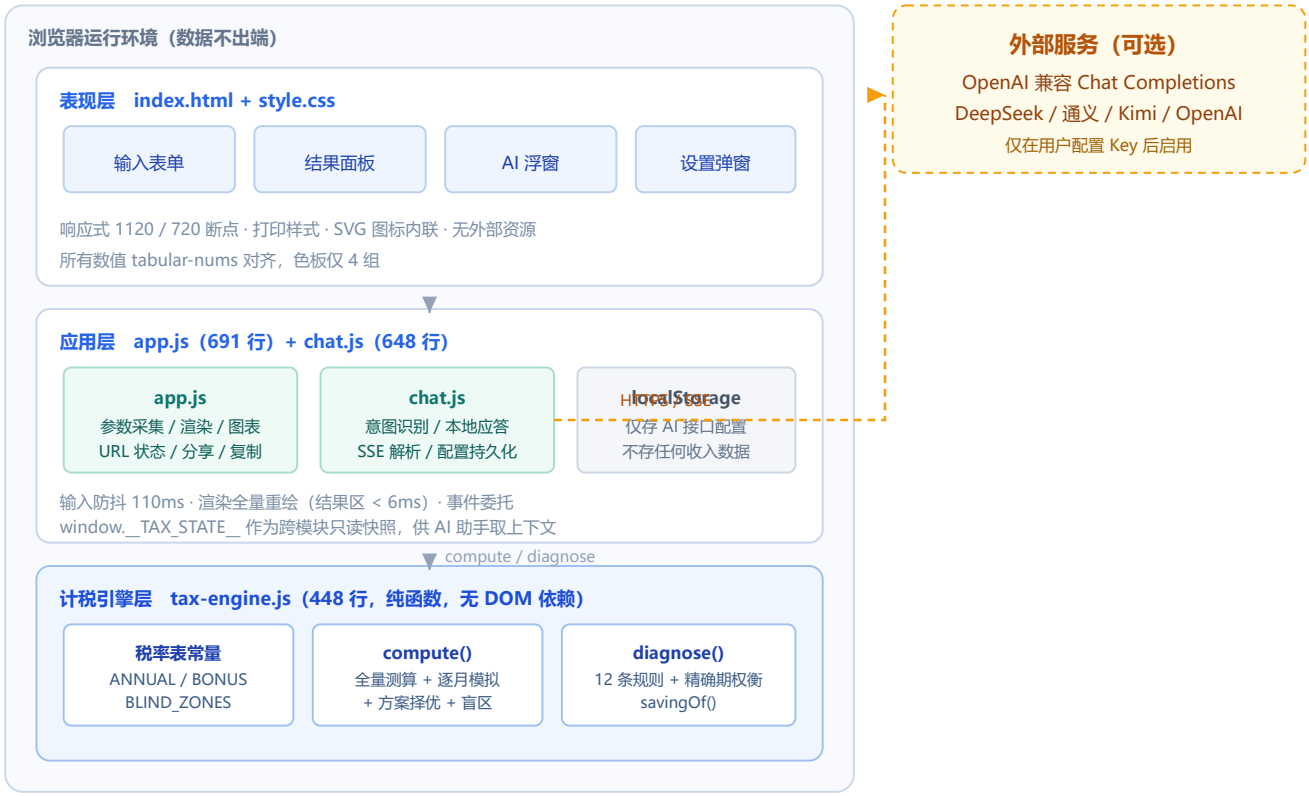
状态放 URL 而非服务端

分享方案时把 18 个参数 Base64 编码进 hash。零存储成本、零账号体系，代价是链接较长（约 500 字符），且参数对持有链接者可见——这一点已在安全自评中登记为风险项。

02 总体架构

四层结构，自下而上为：**计税引擎层**（纯函数、无 DOM）→ **应用层**（状态与渲染）→ **表现层**（DOM 与样式）→ **外部服务**（可选 AI）。层间单向依赖，引擎层可被 Node 直接单测。

2.1 分层架构



依赖方向：表现层 → 应用层 → 引擎层，单向；引擎层不反向依赖任何上层，可独立在 Node 中运行与断言。
跨模块通信仅通过 window.TaxEngine / TaxApp / TaxChat 三个命名空间对象，无全局变量污染。

图 2-1 系统分层架构。虚线为可选出网路径，需用户显式配置凭据后才建立

2.2 模块职责与边界

模块	行数	职责	对外接口	明确不做

tax-engine.js	448	税率表、税额计算、年终奖择优、盲区检测、逐月预扣模拟、缺口诊断	`TaxEngine.compute`
			`TaxEngine.diagnose`
			`TaxEngine.savingOf`
				不读写 DOM、不读表单、不碰 localStorage、不发网络请求

app.js	691	表单采集与截断、调用引擎、渲染六大区块、图表绘制、URL 状态编解码、分享与复制	`TaxApp.state` `TaxApp.recalc`	不含任何税法常量，所有金额口径都来自引擎
chat.js	648	会话管理、上下文快照注入、本地意图识别、API 调用与 SSE 解析、错误归因、配置持久化	`TaxChat.open / send` `TaxChat.onStateChange`	不自行计算税额——所有数字取自已渲染的结果快照
style.css	1,174	设计系统（色板、字号、间距、阴影）、布局、组件样式、响应式与打印	—	不含内联样式之外的动态逻辑；不依赖预处理器

> **这个边界带来的直接收益**：引擎层被完全剥离后，可以在 Node 中用 `vm` 沙箱加载并对 33 条断言逐项验证（见第 8 章），不需要 jsdom、不需要浏览器，测试执行时间 0.34 秒。

2.3 数据流

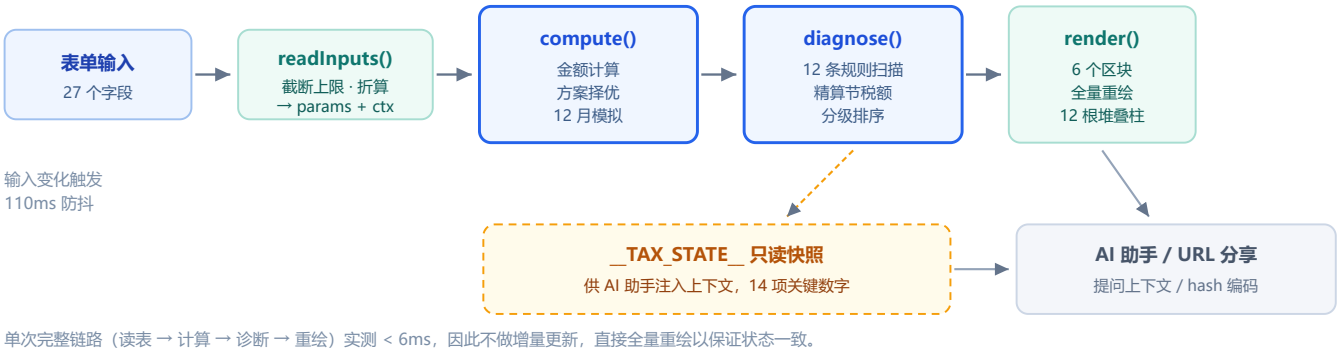


图 2-2　单次交互的数据流。诊断依赖引擎输出，AI 助手依赖渲染后的只读快照，形成三级单向依赖

03 模块设计

3.1 引擎层：纯函数设计

引擎层的三条设计原则：

1. **无副作用**——`compute(params)` 不修改入参，不读写任何外部状态，相同入参必得相同输出；
2. **无环境依赖**——不使用 `document` / `window` 的浏览器专属能力，仅挂载一个命名空间对象；
3. **常量前置**——税率表、扣除标准、盲区区间全部以常量数组定义在文件顶部，政策调整时只改表不改算法。

```
// 政策变更时唯一需要修改的地方
var ANNUAL = [
  { up: 36000,  rate: 0.03, qd: 0 },
  { up: 144000, rate: 0.10, qd: 2520 },
  { up: 300000, rate: 0.20, qd: 16920 },
  { up: 420000, rate: 0.25, qd: 31920 },
  { up: 660000, rate: 0.30, qd: 52920 },
  { up: 960000, rate: 0.35, qd: 85920 },
  { up: Infinity, rate: 0.45, qd: 181920 }
];

// 查表统一走同一个函数，避免各处重复写 if-else 导致口径漂移
function pick(amount, table) {
  for (var i = 0; i < table.length; i++) {
    if (amount <= table[i].up) return table[i];
  }
  return table[table.length - 1];
}
function taxBy(amount, table) {
  if (!(amount > 0)) return 0;
  var b = pick(amount, table);
  return Math.max(0, amount * b.rate - b.qd);
}
```

3.2 应用层：状态与渲染

应用层不做增量 DOM 更新，而是**每次输入全量重绘六个区块**。这个决策基于实测：单次「读表 → 计算 → 诊断 → 重绘」耗时小于 6ms，远低于一帧 16ms 的预算；而全量重绘换来的是**渲染结果与状态永远一致**，不会出现「改了月薪但年终奖对比没更新」这类经典的增量更新 bug。

渲染区块	实现要点
主指标 Hero	大字金额 + 税负率 +「扣除项已帮您省下 ¥X」（X 由「零扣除基线」与当前结果作差得出）
税率档刻度条	7 段等高块，按 `ANNUAL[i].rate === 当前税率` 高亮，无 SVG 依赖
月度现金流	12 根柱，每根内部三段（社保公积金 / 个税 / 到手）按百分比高度堆叠；数值为 0 的段宽度置 0，避免误读
收入 → 扣除 → 应税	5 行水平条，宽度 = 该项 / 税前总收入，末尾追加应纳税所得额与全年个税两行
诊断建议	按「风险 > 重点 > 提示 > 科普」再按节税额降序；汇总时跳过 `realized` 项
测算明细	四组（收入 / 扣除 / 计税 / 结果）分行表，可一键复制为纯文本

3.3 AI 助手层：双模式降级

模式 A • 本地规则引擎（默认）

按关键词匹配 8 类意图（年终奖 / 公积金 / 扣除缺口 / 税负原因 / 汇算 / 个人养老金 / 计算原理 / 兜底画像），应答模板中嵌入当前测算的真实数字。零网络、零成本、零延迟。

价值：即使永远不配 Key，助手也是可用的，不会退化成摆设。

模式 B • OpenAI 兼容接口

标准 Chat Completions，`stream: true` 逐块解析 SSE。系统提示词注入测算快照 + 6 条回答约束（结合数字、标注依据、≤220 字、拒绝违法建议、不编造）。

降级：接口失败时在气泡内展示归因（CORS / 401 / 404 / 429），并自动附上本地答案。

04 算法说明

本章是本系统的技术核心。四个算法依次解决：**算多少**（综合所得税额）、**怎么算更省**（年终奖择优）、**怎么发不亏**（盲区求解）、**为什么逐月变少**（累计预扣）。最后是贯穿全系统的**精确期权衡**原则。

4.1 综合所得应纳税额

采用「速算扣除数」法，等价于分段累进，但只需一次乘减：

```
应纳税所得额 = 综合所得收入额 − 60,000 − 三险一金 − 专项附加扣除 − 其他扣除
应纳个税     = 应纳税所得额 × 适用税率 − 速算扣除数

其中 适用税率 与 速算扣除数 由「应纳税所得额」查七级表确定。
```

速算扣除数的本质，是把前几档的税额差预先折算成一个常数。以第 2 档（36,000–144,000，10%）为例：

```
速算扣除数 = Σ(每一档上限 × (本档税率 − 上一档税率))
           = 36,000 × (10% − 3%) = 2,520 ✓
第 3 档：36,000×(10%−3%) + (144,000−36,000)×(20%−10%) = 2,520 + 10,800 = 13,320？
```

注意：上式是「按档累进差额」的推导思路，但法定速算扣除数（16,920）另有简单来源——它等于**上一档上限按本档税率计税的税额**：`144,000×20% − 16,920 = 11,880`，恰等于 `144,000×10% − 2,520 = 11,880`。**在临界点上两种算法的税额必须连续**，这正是实现中最容易出错的地方，也是我们把它写进单元测试的原因。

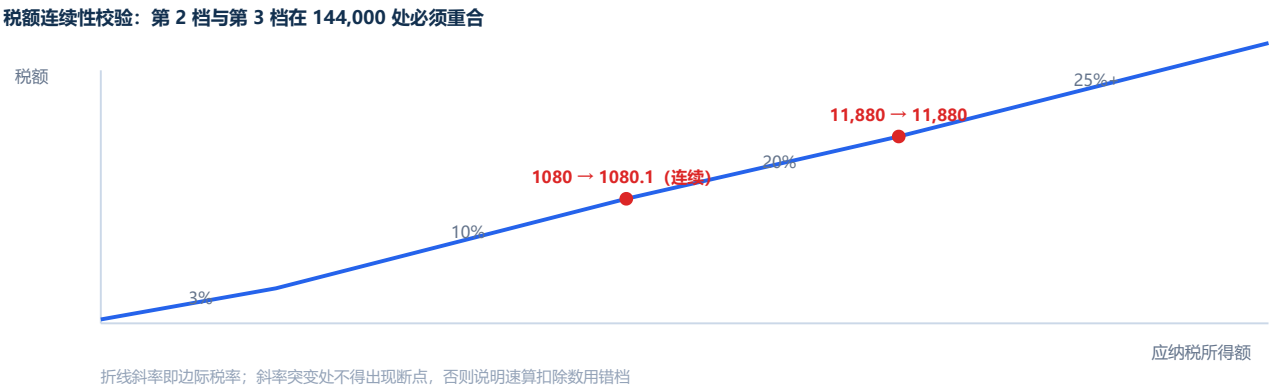


图 4-1　七级累进的税额连续性。单元测试在六个临界点逐点断言「左极限 = 右极限 ±0.1」

4.2 年终奖计税：两种口径与自动择优

年终奖（全年一次性奖金）有两条互斥的计税路径，且**每年只能选择一次**：

口径	计算方式	特点
单独计税	奖金 ÷ 12 查月换算税率表定档 → `税额 = 奖金 × 该档税率 − 该档速算扣除数`	不与工资合并，不会把工资推高到更高档；但跳档时税额突变
并入综合	奖金全额并入综合所得收入额，重新计算应纳税所得额后按七级表计税	可充分利用未用尽的低档空间；但可能把整体推入高档

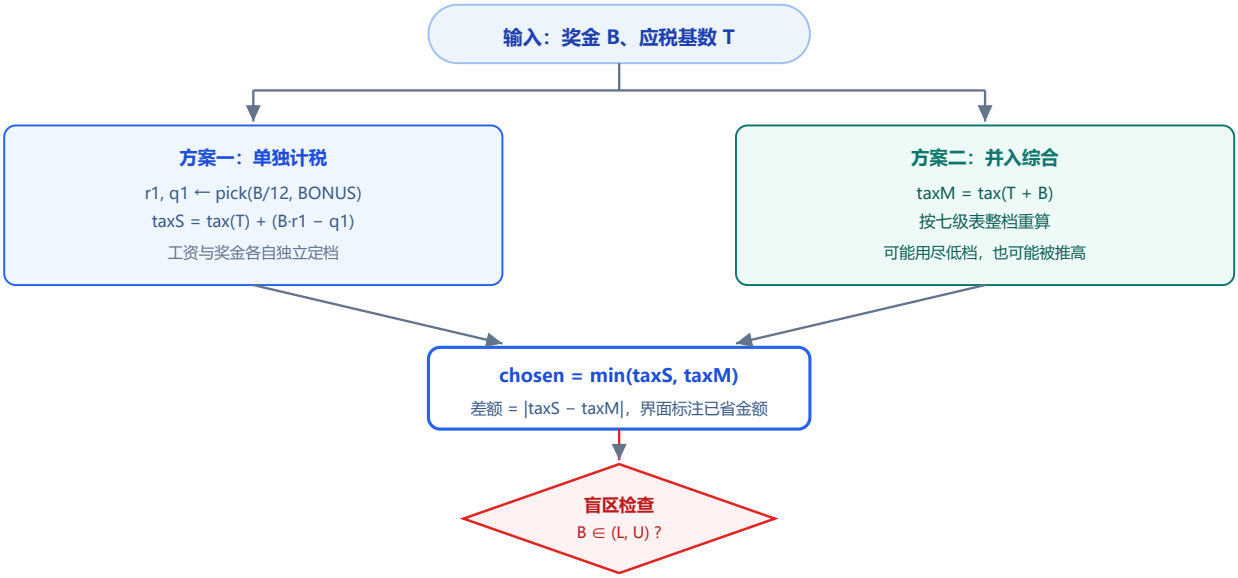


图 4-2　年终奖计税决策流程。注意盲区检查在择优之后独立进行——两者回答的是不同问题

4.3 「多发少得」盲区：解方程而非硬编码

单独计税的定档依据是 `B/12`，但计税基数是 `B` 全额。这导致在档位临界点上，税率跃升而速算扣除数只增加一点点，税后收入出现**断崖**。

以第一个临界点为例：奖金 36,000 元时税额 1,080 元，税后 34,920 元；奖金多发 1 元变成 36,001 元，税额跳到 3,390.1 元，**税后反而只有 32,610.9 元——发得越多，拿得越少。**

求解盲区上界

盲区的上界不是拍脑袋定的「大约 3.8 万」，而是让**多发后的税后收入追平临界点税后收入**的那个奖金值。设临界点 L、其税后收入 `A(L) = L − tax(L)`，下一档税率 `r`、速算扣除数 `qd`，求解：

```
B − (B·r − qd) = A(L)
B·(1 − r)      = A(L) − qd
        B      = (A(L) − qd) / (1 − r)

代入 L = 36,000：A = 36,000 − 1,080 = 34,920；r = 10%，qd = 210
B = (34,920 − 210) / 0.9 = 38,566.67 ✓
```

同一套公式代入其余五个临界点，得到的结果与官方口径**完全一致**——这一点已写入单元测试（`tests/engine.test.js` 逐点断言，误差容忍 0.01 元）：

序号	临界点 L	盲区上界 U	区间宽度	最大税后损失
1	36,000	38,566.67	2,566.67	2,310.00
2	144,000	160,500.00	16,500.00	13,200.00
3	300,000	318,333.33	18,333.33	13,750.00
4	420,000	447,500.00	27,500.00	19,250.00
5	660,000	706,538.46	46,538.46	30,250.00
6	960,000	1,120,000.00	160,000.00	88,000.00

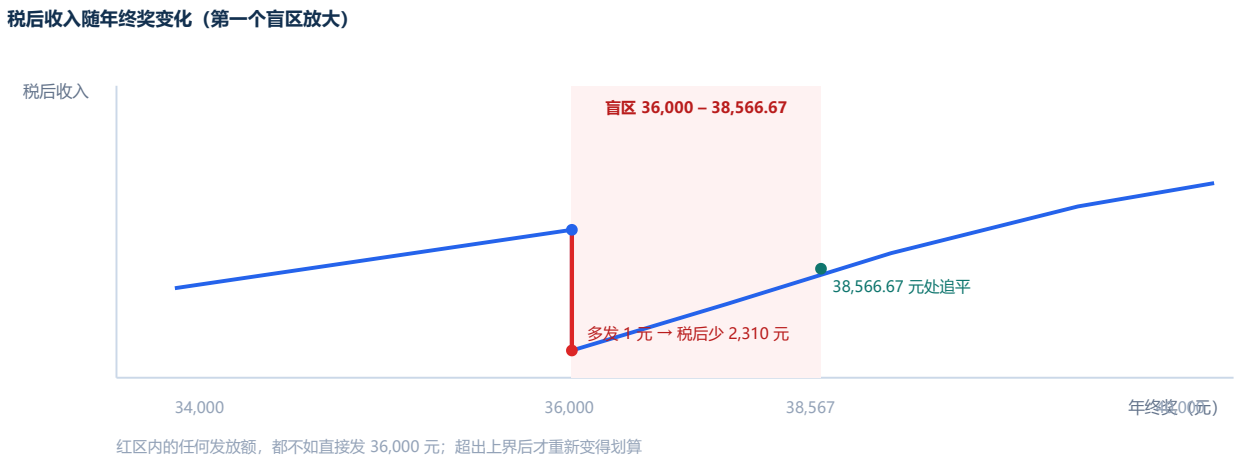


图 4-3　盲区的几何解释。断崖来自税率跃升，追平点由 4.3 的解方程结果决定

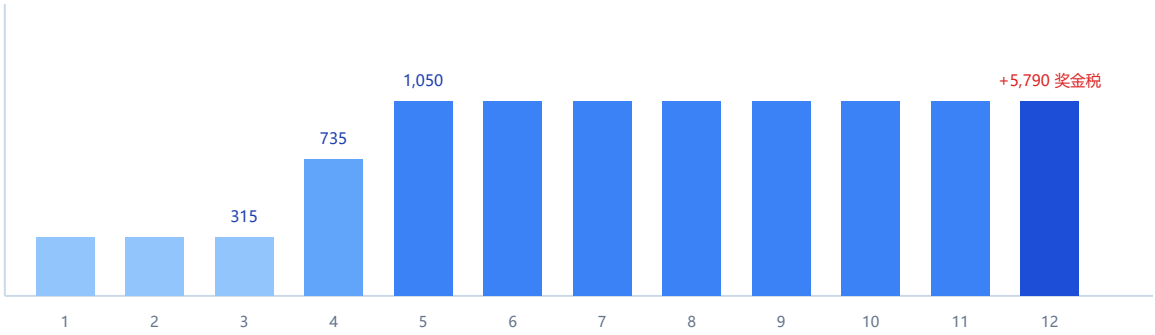
4.4 累计预扣：为什么下半年到手会变少

工资薪金按月预扣，但算法不是「每月算一次」，而是**每月重算年初至今的累计应纳税额，再减去已缴部分**：

```
累计应纳税所得额 = 累计收入 − 5,000×月数 − 累计三险一金 − 累计专项附加 − 累计其他扣除
本期应预扣税额   = (累计应纳税所得额 × 预扣率 − 速算扣除数) − 累计已预扣税额
```

由于累计应纳税所得额逐月递增，跨越档位后**新增部分按更高税率计税**，因此月度税额呈阶梯上升、到手金额阶梯下降。系统对 12 个月逐月模拟，把这条曲线画出来，用户一眼就能对上自己的工资条。

示例：月薪 20,000、三险一金 4,500/月、无专项附加时的逐月预扣



1–3 月累计所得仍在 3% 档；第 4 月跨入 10% 档；第 5 月起稳定在 1,050/月。全年预扣合计 15,870 元 = 综合所得 10,080 + 奖金 5,790。

图 4-4 逐月预扣的阶梯。单元测试断言：12 个月税额之和恒等于全年应纳个税

4.5 精确期权衡：节税额不能乘边际税率

诊断每条建议都要回答「这么做能省多少钱」。最直觉的做法是 `扣除额 × 边际税率`，但这是**错的**——扣除会把应纳税所得额往下推，跨档后只有一部分按高税率抵扣。

```
朴素算法（错误）：saving = d × 边际税率
精确算法（采用）：saving = tax(T) − tax(max(0, T − d))

反例：T = 150,000（20% 档），新增扣除 d = 10,000
  朴素：10,000 × 20% = 2,000
  精确：tax(150,000) − tax(140,000) = 13,080 − 11,480 = 1,600
  误差：+400 元（朴素算法高估 25%）
```

为什么必须精算：诊断结果会引导用户行动（去补缴公积金、开个人养老金账户）。如果告诉他能省 2,000 元而实际只省 1,600 元，这个工具的信任度就崩了。这条规则已被固化为单元测试——`engine.test.js` 专门断言「精确值 1,600 ≠ 朴素值 2,000，且精确值更小」。

4.6 诊断规则的统一结构

12 条规则共用同一套输出契约，因此可以统一排序、统一渲染、统一汇总：

字段	类型	含义
`key`	string	规则标识（housing / elderly / child / edu / fund / pension / medical / bracket / blind / mode / high / socialbase）
`level`	enum	danger 风险 / high 重点 / mid 提示 / info 科普，决定图标与配色
`title`	string	一句话结论，如「距降档只差 3,200 元扣除额」
`detail`	string	展开说明，含具体数字与操作方式
`saving`	number	预计可节税额，由 `savingOf()` 精算
`savingLabel`	string?	金额口径文案，默认为「预计可节税」；盲区项为「税后多拿」
`realized`	boolean?	是否已被系统自动优化。为真时汇总不计入，避免虚增节税空间
`basis`	string	测算依据，如「按独生子女 3000 元/月测算」，保证可解释

> **设计要点：** `realized` 字段解决了一个容易忽略的正确性问题——年终奖择优本身能省 4,410 元，但如果把它也算进「你还能省多少」，用户会以为还有 4,410 元可以再省一次。汇总时排除该项，是这类工具最容易犯又最难发现的错误。

05 数据模型

5.1 输入参数 params

字段	类型	说明
`monthlySalary`	number	税前月薪，内部按 ×12 作为全年工资薪金
`bonus`	number	全年一次性奖金
`otherIncome`	number	其他综合所得**收入额**（劳务报酬按 80% 折算后的值）
`socialMonthly`	number	社保个人月缴 = 基数 × 费率
`fundMonthly`	number	公积金个人月缴 = 基数 × 比例
`addMonthly`	number	专项附加扣除中**按月申报**的部分（子女教育、婴幼儿、学历继续教育、房贷/租金、赡养老人）
`addAnnual`	number	专项附加扣除中**仅汇算扣除**的部分（大病医疗、职业资格证书 3,600）
`otherAnnual`	number	个人养老金 + 商业健康险
`bonusMode`	enum	auto / separate / merge

为什么要把 addMonthly 与 addAnnual 分开

这不只是分类，而是**两个不同计税口径的分界线**：按月申报的项目会进入月度预扣模拟，从而影响每月到手；而大病医疗、职业资格证书只能在年度汇算时扣除，**不体现在月度预扣里。**正是这个差值造成了「汇算可退税」。

```
汇算差额 = 月度预扣合计 − 全年应纳个税
        > 0 → 可退税（典型：有 2 万大病医疗扣除，预扣时没享受到）
        < 0 → 需补税
```

5.2 计算结果 result（关键字段）

分组	字段
输入回显	`inputs.*`
收入构成	`gross{ salaryYear, otherIncome, bonus, comprehensive, total }`
扣除明细	`deduction{ basic, specialYear, socialYear, fundYear, addMonthlyYear, addAnnual, additionalYear, otherAnnual, total }`
计税结果	`taxable, rate, quickDeduction, comprehensiveTax, bonusTax, totalTax, marginalRate`
方案对比	`compare{ separate, merge, best, diff, chosen, bonusIncluded }`
到手测算	`afterTaxAnnual, monthlyNetAvg, effectiveRate`
逐月模拟	`months[12]{ month, gross, social, fund, tax, net }`
汇算	`withheld, settlement`
盲区	`blindZone{ inZone, lower, upper, loss }`

5.3 跨模块状态

三个命名空间对象构成唯一的状态通道，避免隐式全局变量：

```
window.TaxEngine   // 常量与方法，只读
window.TaxApp      // { state, recalc, yuan, money, pct }
window.TaxChat     // { onStateChange, open, send }
window.__TAX_STATE__ // 渲染后的只读快照，AI 助手据此取上下文

// app.js 每次渲染末尾推送给助手，助手不需要重新读表单
if (window.TaxChat && window.TaxChat.onStateChange) window.TaxChat.onStateChange(state);
```

06 安全设计

本系统的安全设计遵循一条主线：**把攻击面压缩到「一个不需要信任任何服务器」的程度**。因为收入数据从不上传，绝大多数 Web 应用的高危风险（拖库、越权、服务端注入、日志泄露）在本系统中**结构性不存在**，而不是被修补掉。剩余风险集中在客户端本地与可选出网路径上。

6.1 信任边界



图 6-1 信任边界。整个系统只有一个出网口，且该出口由用户显式开启

6.2 数据安全

威胁	等级	设计对策
收入数据泄露	—	结构性消除：无后端、无数据库、无日志。数据不离开浏览器，攻击者即使攻破静态托管服务器也拿不到任何用户数据
服务端越权访问	—	不存在服务端业务接口，无越权面
埋点与第三方统计	—	零埋点、零 Cookie、零第三方脚本（已有测试断言「不加载任何 CDN 资源」）
分享链接外泄	中	参数编码在 hash（`#s=`），**不会随 HTTP 请求发送到服务器**；但持有链接者可见参数，属已知风险，界面与文档均提示「分享前确认」
共用设备残留	中	收入数据不落盘；但 localStorage 中的 API Key 会残留，提供「清除」按钮并在文档中提示

6.3 凭据管理

- **不自建密钥池**：系统不内置任何厂商 Key，用户自备。这从根本上避免了「服务端保管大量用户密钥」这一高危资产。

- **隔离存储**：Key 仅存 `localStorage['taxai.cfg.v1']`，与收入数据分离，且从不写入 URL、不进分享链接、不出现在任何导出文本中。
- **输入即密文**：Key 输入框 `type="password"`，`autocomplete="off"`，避免浏览器密码管理器误存。
- **可撤销性**：文档建议用户使用**限额、可随时吊销**的 Key；泄露后吊销即可，影响面可控。

6.4 AI 内容安全

大模型返回内容是本系统唯一的「不可信输出」，采用三重防护：

1. **提示词约束**——系统提示词明确要求：只讨论合法合规的税前扣除与计税方式选择；对虚开发票、虚假申报等请求明确拒绝并说明风险；不确定的数字不得编造，引导用户以申报和汇算结果为准。
2. **渲染转义**——模型输出先 `escapeHtml()` 转义 `& < > "`，再用受限的 Markdown 子集（粗体、行内代码、无序列表）还原样式，**不执行任何脚本、不渲染图片与外链**，杜绝 XSS 与内容注入。
3. **数据最小化**——注入上下文的只有 14 项数值（月薪、奖金、扣除、税额、税率等），不含姓名、证件号、单位、银行账户。

6.5 输入与实现健壮性

```
// 所有外部输入统一收口，避免 NaN / Infinity / 负数穿透到计算层
function num(v) {
  var n = parseFloat(v);
  return isFinite(n) ? n : 0;          // NaN / Infinity → 0
}
// 计算入口再做一次非负钳制
var monthlySalary = Math.max(0, num(p.monthlySalary));

// 法定上限在读取时截断，而非依赖用户自觉
detail.pension   = clamp(num('pension'), 0, 12000);
detail.healthIns = clamp(num('healthIns'), 0, 2400);
detail.medical   = clamp(num('medicalAmount'), 0, 80000);
elderlyShare     = clamp(num('elderlyShare'), 0, 1500);
```

此外，分解计算全过程不使用 `eval` / `new Function` / `innerHTML` 拼接用户输入；结果区所有数值均由 `Number.toLocaleString` 格式化后写入，不存在注入路径。

6.6 部署与传输安全

项	要求
HTTPS	必须启用。否则浏览器会以混合内容拦截对 AI 接口的请求，且分享链接在传输中可被篡改
响应头	建议追加 `Content-Security-Policy: default-src 'self'; connect-src 'self' https:; img-src 'self' data:; script-src 'self'` 与 `X-Content-Type-Options: nosniff`（详见安全自评的改进建议）
供应链	零第三方依赖，无 npm 包、无 CDN，不存在依赖投毒与版本漂移风险
完整性	静态文件可做发布前哈希校验；因无构建步骤，发布物与源码一一对应，可逐行审计

07 性能与兼容性

7.1 性能指标

指标	实测	说明
首屏内容渲染	< 1 s	单 HTML + 1 CSS + 3 JS，无网络请求依赖，本地直接打开即可
单次完整重算 + 重绘	< 6 ms	读表 → compute → diagnose → 六区块渲染
输入响应	110 ms 防抖	连续输入时只在停顿后计算一次，避免抖动
页面总资源	约 120 KB	未压缩（HTML 22 KB + CSS 26 KB + JS 72 KB），gzip 后约 32 KB
测试执行	0.34 s	33 条断言，无浏览器启动开销

7.2 兼容性矩阵

环境	支持	备注
Chrome / Edge	✓	主要验证环境（Chromium 内核）
Safari（macOS / iOS）	✓	已适配 `-webkit-` 前缀与 `100vh` 问题
微信内置浏览器	✓	分享链接的主要落地场景，用 ES5 语法保证兼容
Firefox	✓	未使用 Chromium 专有 API
IE 11	部分	核心计算可用；flex 与 CSS 变量降级，视觉不完整

7.3 响应式策略

- **≥1120px**：左侧输入固定 396px 吸附，右侧结果自适应，最大化「改一个数、立刻看到变化」的效率；
- **720–1120px**：单列纵向排列，输入在上结果在下；
- **<720px**：移动布局，AI 面板转全宽，图表浮层关闭（触屏无 hover），触控热区 ≥38px；
- **打印**：隐藏顶栏按钮、AI 浮窗、政策条，输出干净的结果页。

08 测试与验证

测试的核心目标不是覆盖率数字，而是**把「税法口径」变成可执行的断言**。任何人改动税率表或算法，都会在 0.34 秒内被 33 条断言拦住。

8.1 三层测试策略

层次	用例数	验证内容
引擎单测 engine.test.js	16	七档税率表逐个边界、年终奖按月换算定档、六个盲区上下界、盲区内外判定、自动择优、逐月预扣合计恒等于全年税额、仅汇算扣除项产生退税、精确期权衡与朴素算法差异、诊断规则触发与汇总去重
验收用例 acceptance.test.js	8	PRD 第 8.1 节 AC-01 ~ AC-08，期望值精确到元，覆盖典型白领、扣除增量、高收入并入更优、盲区命中与越界、其他综合所得、非法输入
静态契约 smoke.test.js	9	关键元素 id 齐全、脚本加载顺序、资源文件真实存在、零 CDN 外部依赖、法定上限写在控件上、`[hidden]` 强制生效、响应式与打印断点、免责文案存在

8.2 引擎如何在 Node 中被测试

引擎挂载在 `window` 上，但本身不依赖浏览器。测试用 Node 的 `vm` 模块构造一个只有 `window` 的沙箱，把源码当作普通脚本执行，随后从沙箱取回引擎对象：

```
const vm = require('node:vm');
const code = fs.readFileSync('assets/js/tax-engine.js', 'utf8');

const sandbox = { window: {} };
vm.createContext(sandbox);
vm.runInContext(code, sandbox, { filename: 'tax-engine.js' });

const E = sandbox.window.TaxEngine;
E.compute({ monthlySalary: 20000, bonus: 60000, /* ... */ });
```

这样既不需要 jsdom，也不需要把引擎改造成 CommonJS 模块——**生产代码为零测试适配付出任何代价。**

8.3 关键断言示例

```
test('年终奖盲区：六个区间的上下界与官方口径一致', () => {
  const expected = [
    [36000, 38566.67], [144000, 160500], [300000, 318333.33],
    [420000, 447500],  [660000, 706538.46], [960000, 1120000]
  ];
  E.BLIND_ZONES.forEach((z, i) => {
    near(z.lower, expected[i][0]);
    near(z.upper, expected[i][1]);   // 误差容忍 0.01 元
  });
});
```

```
test('累计预扣法：逐月税额合计等于全年综合所得税额', () => {
  const r = E.compute(BASE);
  const sum = r.months.reduce((s, m) => s + m.tax, 0);
  near(sum, r.comprehensiveTax + r.bonusTax);
});
```

8.4 视觉与回归验证

除自动化测试外，开发过程使用无头浏览器做像素级检查：以 1440×2800 与 520×1800 两种视口截图，再用图像处理裁切局部放大核对排版；同时注入探针脚本统计 `documentElement.scrollWidth` 与越界元素，确认无横向溢出。

> **踩过的坑（已沉淀为测试）**：① `[hidden]` 属性会被 `.modal{display:grid}` 之类的类选择器覆盖，导致弹窗永远显示——因此强制 `[hidden]{display:none!important}` 并加断言；② 图表悬浮提示采用 `position:absolute` + `white-space:nowrap`，在窄屏会造成横向滚动——移动端直接关闭浮层。这两条都写进了 `smoke.test.js`。

09 部署架构

9.1 部署拓扑

系统只有一个可部署单元：**静态文件目录。**没有应用服务器、没有数据库、没有反向代理之外的任何中间件。



图 9-1　部署拓扑。虚线框表示「不存在」或「可选」

9.2 发布流程

```
1. 本地修改 → npm test（33 条断言，0.34 s）
2. 无头浏览器截图核对（1440 / 520 两档视口）
3. 推送仓库 → CI（Node 18/20/22 三版本跑同一套测试）
4. 部署：上传目录 / 触发平台构建
5. 部署后自检 8 项（首页 / 数值 / 样式 / 交互 / 扣除 / 图表 / 助手 / 分享）
```

9.3 缓存策略

文件名未做内容哈希，因此静态资源的 `max-age` 不宜过长（建议 1 小时并带 `must-revalidate`），HTML 走 `no-cache`。这样既避免每次刷新都重新下载，又能保证发布后一小时内新版本生效。

10 演进规划

10.1 已识别的技术债

编号	问题	影响	处理方案
TD-1	政策参数硬编码在源码中	中	抽为按年度索引的配置对象 `POLICY[year]`，界面年度切换即切换税率表与扣除标准
TD-2	其他综合所得统一按 80% 口径	中	拆为劳务报酬 / 稿酬 / 特许权三类分别录入，各自按 80% / 56% / 80% 折算
TD-3	月度模拟假设等额发放	低	支持「年度总额 + 调薪月份」或逐月录入
TD-4	社保基数未校验当地上下限	中	内置主要城市社保基数上下限表，超限时提示并按上限计算
TD-5	append 式渲染，无虚拟 DOM	低	当前规模无需优化；若区块继续增加，改为分区局部更新

10.2 版本路线

版本	技术增量
v1.1	夫妻扣除分配优化——把「各扣 50% / 一方 100%」的组合变成一个**最优化问题**：枚举扣除归属组合，以家庭合计税额最小为目标求解；附多年收入对比与 PDF 导出
v1.2	政策参数配置化（TD-1）、综合所得分类录入（TD-2）、社保上下限（TD-4）
v2.0	多方案增发测算（年终奖与月薪结构调整的联合优化）；企业版批量测算，仍然纯本地处理以保证数据不出端

10.3 一个不做的事

> **不接入官方申报接口。**一旦系统能代替用户申报，就必须承载身份凭据、承担申报准确性的法律责任，并接受等保与个人信息保护合规审查。这与「数据不出端、零信任服务器」的核心设计相冲突。本工具的定位始终是**决策辅助**，而非**申报代理**。

本文档描述 v1.0.0 实现。涉及税法口径的内容依据《中华人民共和国个人所得税法》及财政部、税务总局相关公告，具体见《需求文档》附录 A。文档中的性能数据来自开发机实测，实际表现受设备与浏览器影响。